

## Class Test

### Q1 Default and Parameterized Constructors

Write a C++ program that defines a class Book with attributes title, author, and price. Implement:

- A. A default constructor that initializes all attributes to default values ("Unknown", "Unknown", 0.0).
- B. A parameterized constructor that accepts values for all three attributes.
- C. A display() member function to print the book details.

In main(), create one object using each constructor and display its details.

Your Code

Input

```

3 using namespace std;
4 {
5     class book{
6         string title, author;
7         double price;
8     public:
9         book(){
10             title = "unknown";
11             author = "unknown";
12             price=0.00;
13         }
14         I
15         book(string t, string a, double p){
16             title=t;
17             author=a;
18             prices=p;
19         }
20         void display(){
21             cout<<"title"<<title;
22             cout<<"author"<<author;
23             cout<<"prices"<<price;
24         }
25     };
26     int main(){
27         book b1;
28         cout<<"book"<<endl;
29         b1.display();
30         cout<<endl;
31         book b2("C++ Programming", "bjarme stroustrup", 500.50);

```

Output

### Visible Test Cases

Test Case 1

Test Case 2

Test Case 3

## Q2 Constructor Overloading

Write a C++ program that defines a class Rectangle with the following constructors:

- A. A default constructor that sets length and breadth to 1.
- B. A single-parameter constructor that creates a square (length = breadth).
- C. A two-parameter constructor that sets both length and breadth.

Include a member function area() to compute the rectangle's area. In main(), create three objects using each constructor and print their areas.

Your Code

Input

```

1 #include <iostream>
2 using namespace std;
3
4 class Rectangle {
5     int length, breadth;
6
7 public:
8     Rectangle() {
9         length = 1;
10        breadth = 1;
11    }
12
13    Rectangle(int side) {
14        length = side;
15        breadth = side;
16    }
17
18    Rectangle(int l, int b) {
19        length = l;
20        breadth = b;
21    }
22
23    int area() {
24        return length * breadth;
25    }
26 };
27
28 int main() {
29     Rectangle r1;

```

stdin input

Output



### Visible Test Cases

Test Case 1

Test Case 2

Test Case 3

### Q3 Copy Constructor with Dynamic Memory

Write a C++ program that defines a class Student containing a dynamically allocated character array for the student's name and an integer for marks. Implement:

- A. A parameterized constructor that allocates memory and copies the name.
- B. A deep copy constructor that allocates new memory for the copied object.
- C. A destructor that releases the allocated memory.
- D. A display() function to print the student's details.

In main(), create a Student object, create another object using the copy constructor, and modify the second object's name to demonstrate that the original remains unaffected.

```
1 #include <iostream>
2 #include <cstring>
3 using namespace std;
4
5 class Student {
6     char* name;
7     int marks;
8
9 public:
10     Student(const char* n, int m) {
11         name = new char[strlen(n) + 1];
12         strcpy(name, n);
13         marks = m;
14     }
15
16     Student(const Student& s) {
17         name = new char[strlen(s.name) + 1];
18         strcpy(name, s.name);
19         marks = s.marks;
20     }
21
22     ~Student() {
23         delete[] name;
24     }
25
26     void display() {
27         cout << "Name: " << name << endl;
```

## Class Test

### Q4 Destructor and Resource Management

Write a C++ program that defines a class FileHandler which simulates opening and closing a file. Implement:

- A. A constructor that prints "File <name> opened".
- B. A destructor that prints "File <name> closed".

In main(), create three FileHandler objects inside a block {} and observe the order in which constructors and destructors are invoked. Print a message after the block ends to confirm the destruction order.

```
1 #include <iostream>
2 using namespace std;
3
4 class FileHandler {
5     string name;
6
7     public:
8         FileHandler(string n) {
9             name = n;
10            cout << "File " << name << " opened" << endl;
11        }
12
13        ~FileHandler() {
14            cout << "File " << name << " closed" << endl;
15        }
16 };
17
18 int main() {
19     {
20         FileHandler f1("A");
21         FileHandler f2("B");
22         FileHandler f3("C");
23     }
24
25     cout << "Block ended" << endl;
26 }
```

#### Q5 Overloading the + Operator (Complex Numbers)

Write a C++ program that defines a class Complex with real and imaginary parts. Overload the + operator to add two complex numbers and return a new Complex object. Include a display() function to print the result in the form a + bi.

In main(), create two Complex objects, add them using the overloaded + operator, and display the result.

```

1 #include <iostream>
2 using namespace std;
3
4 class Complex {
5     int real, imag;
6
7 public:
8     Complex(int r, int i) {
9         real = r;
10        imag = i;
11    }
12
13    Complex operator+(Complex c) {
14        return Complex(real + c.real, imag + c.imag);
15    }
16
17    void display() {
18        cout << real << " + " << imag << "i" << endl;
19    }
20 };
21
22 int main() {
23     Complex c1(3, 4);
24     Complex c2(5, 6);
25
26     Complex c3 = c1 + c2;
27
28     c3.display();

```

stdin input

Output

### Visible Test Cases

Test Case 1

Test Case 2

Test Case 3